Response to Feedback: Project 5 – DATA 612 (Recommender Systems)

Reflections and Refinements on Spark ALS Implementation

Sheriann McLarty

Thanks so much for your feedback on Project 5 for DATA 612! I really appreciate the chance to take a closer look at my work and dig deeper into how Spark's Alternating Least Squares (ALS) ticks.

To provide you with more technical details, I've added a more precise explanation of how ALS works in Spark. Essentially, the algorithm breaks down the large user-item ratings matrix into two smaller ones: one for users and one for items. The idea is to learn these matrices so that, when multiplied, they predict ratings as closely as possible to the real ones. Spark does this by minimizing the regularized squared error, flipping back and forth between updating user factors (while fixing items) and updating item factors (while fixing users). It repeats this process until it converges, or it hits the max number of iterations.

For my project, I set up Spark's ALS with some key parameters: maxIter=10 to keep training efficient, regParam=0.1 to help avoid overfitting, and coldStartStrategy='drop' to dodge any NaN values when evaluating. Even though I ran Spark just on my local machine, its distributed system still let me parallelize the training, which was a huge help even without a big cluster.

I also documented the main hiccups I encountered, such as data casting problems and platform quirks. Running Spark locally actually made it easier for me to troubleshoot and maintain control over the entire process.

Thanks again for pushing me to dive a bit deeper. I've incorporated all this extra detail into my final write-up and appreciate the learning experience that your feedback helped create.